

```

prev = l[0]
print("prev = l[0]", "\t\tprev:", prev)
del l[0]
print("del l[0]", "\t\tl:", l)
for item in l:
    if prev == item:
        print("Duplicate of", prev, "found")
    print("if prev == item:", "\t\tprev:", prev, "\t\titem:", item)
    prev = item
    print("prev = item", "\t\tprev:", prev, "\t\titem:", item)

```

with the output being:

l = [4, 5, 7, 8, 9, 1, 0, 7, 10]	l: [4, 5, 7, 8, 9, 1, 0, 7, 10]
l.sort()	l: [0, 1, 4, 5, 7, 7, 8, 9, 10]
prev = l[0]	prev: 0
del l[0]	l: [1, 4, 5, 7, 7, 8, 9, 10]
if prev == item:	prev: 0 item: 1
prev = item	prev: 1 item: 1
if prev == item:	prev: 1 item: 4
prev = item	prev: 4 item: 4
if prev == item:	prev: 4 item: 5
prev = item	prev: 5 item: 5
if prev == item:	prev: 5 item: 7
prev = item	prev: 7 item: 7
Duplicate of 7 found	
if prev == item:	prev: 7 item: 7
prev = item	prev: 7 item: 7
if prev == item:	prev: 7 item: 8
prev = item	prev: 8 item: 8
if prev == item:	prev: 8 item: 9
prev = item	prev: 9 item: 9
if prev == item:	prev: 9 item: 10
prev = item	prev: 10 item: 10

The reason I put so many `print` statements in the code was so that you can see what is happening in each line. (By the way, if you can't figure out why a program is not working, try putting in lots of print statements in places where you want to know what is happening.) First the program starts with a boring old list. Next the program sorts the list. This is so that any duplicates get put next to each other. The program then initializes a `prev`(ious) variable. Next the first element of the list is deleted so that the first item is not incorrectly thought to be a duplicate. Next a `for` loop is gone into. Each item of the list is checked to see if it is the same as the previous. If it is a duplicate was found. The value of `prev` is then changed so that the next time the `for` loop is run through `prev` is the previous item to the current. Sure enough, the 7 is found to be a duplicate. (Notice how `\t` is used to print a tab.)

The other way to use `for` loops is to do something a certain number of times. Here is some code to print out the first 9 numbers of the Fibonacci series:

```

a = 1
b = 1
for c in range(1, 10):
    print(a, end=" ")
    n = a + b
    a = b
    b = n

```

with the surprising output:

```
1 1 2 3 5 8 13 21 34
```

Everything that can be done with `for` loops can also be done with `while` loops but `for` loops give an easy way to go through all the elements in a list or to do something a certain number of times.

12 Boolean Expressions

Here is a little example of boolean expressions (you don't have to type it in):

```
a = 6
b = 7
c = 42
print(1, a == 6)
print(2, a == 7)
print(3, a == 6 and b == 7)
print(4, a == 7 and b == 7)
print(5, not a == 7 and b == 7)
print(6, a == 7 or b == 7)
print(7, a == 7 or b == 6)
print(8, not (a == 7 and b == 6))
print(9, not a == 7 and b == 6)
```

With the output being:

```
1 True
2 False
3 True
4 False
5 True
6 True
7 False
8 True
9 False
```

What is going on? The program consists of a bunch of funny looking **print** statements. Each **print** statement prints a number and an expression. The number is to help keep track of which statement I am dealing with. Notice how each expression ends up being either **False** or **True**. In Python false can also be written as 0 and true as 1.

The lines:

```
print(1, a == 6)
print(2, a == 7)
```

print out a **True** and a **False** respectively just as expected since the first is true and the second is false. The third print, **print(3, a == 6 and b == 7)**, is a little different. The operator **and** means if both the statement before and the statement after are true then the whole expression is true otherwise the whole expression is false. The next line, **print(4, a == 7 and b == 7)**, shows how if part of an **and** expression is false, the whole thing is false. The behavior of **and** can be summarized as follows:

expression	result
true and true	true

expression	result
true and false	false
false and true	false
false and false	false

Notice that if the first expression is false Python does not check the second expression since it knows the whole expression is false. Try running `False and print("Hi")` and compare this to running `True and print("Hi")`. The technical term for this is short-circuit evaluation¹

The next line, `print(5, not a == 7 and b == 7)`, uses the `not` operator. `not` just gives the opposite of the expression. (The expression could be rewritten as `print(5, a != 7 and b == 7)`). Here is the table:

expression	result
not true	false
not false	true

The two following lines, `print(6, a == 7 or b == 7)` and `print(7, a == 7 or b == 6)`, use the `or` operator. The `or` operator returns true if the first expression is true, or if the second expression is true or both are true. If neither are true it returns false. Here's the table:

expression	result
true or true	true
true or false	true
false or true	true
false or false	false

Notice that if the first expression is true Python doesn't check the second expression since it knows the whole expression is true. This works since `or` is true if at least one half of the expression is true. The first part is true so the second part could be either false or true, but the whole expression is still true.

The next two lines, `print(8, not (a == 7 and b == 6))` and `print(9, not a == 7 and b == 6)`, show that parentheses can be used to group expressions and force one part to be evaluated first. Notice that the parentheses changed the expression from false to true. This occurred since the parentheses forced the `not` to apply to the whole expression instead of just the `a == 7` portion.

Here is an example of using a boolean expression:

```
list = ["Life", "The Universe", "Everything", "Jack", "Jill", "Life", "Jill"]

# make a copy of the list. See the More on Lists chapter to explain what [:]
# means.
copy = list[:]
# sort the copy
copy.sort()
```

¹ <https://en.wikipedia.org/wiki/Short-circuit%20evaluation>

```
prev = copy[0]
del copy[0]

count = 0

# go through the list searching for a match
while count < len(copy) and copy[count] != prev:
    prev = copy[count]
    count = count + 1

# If a match was not found then count can't be < len
# since the while loop continues while count is < len
# and no match is found

if count < len(copy):
    print("First Match:", prev)
```

And here is the output:

```
First Match: Jill
```

This program works by continuing to check for match `while count < len(copy) and copy[count] is not equal to prev`. When either `count` is greater than the last index of `copy` or a match has been found the `and` is no longer true so the loop exits. The `if` simply checks to make sure that the `while` exited because a match was found.

The other "trick" of `and` is used in this example. If you look at the table for `and` notice that the third entry is "false and false". If `count >= len(copy)` (in other words `count < len(copy)` is false) then `copy[count]` is never looked at. This is because Python knows that if the first is false then they can't both be true. This is known as a short circuit and is useful if the second half of the `and` will cause an error if something is wrong. I used the first expression (`count < len(copy)`) to check and see if `count` was a valid index for `copy`. (If you don't believe me remove the matches "Jill" and "Life", check that it still works and then reverse the order of `count < len(copy) and copy[count] != prev` to `copy[count] != prev and count < len(copy)`.)

Boolean expressions can be used when you need to check two or more different things at once.

12.0.1 A note on Boolean Operators

A common mistake for people new to programming is a misunderstanding of the way that boolean operators works, which stems from the way the python interpreter reads these expressions. For example, after initially learning about "and" and "or" statements, one might assume that the expression `x == ('a' or 'b')` would check to see if the variable `x` was equivalent to one of the strings 'a' or 'b'. This is not so. To see what I'm talking about, start an interactive session with the interpreter and enter the following expressions:

```
>>> 'a' == ('a' or 'b')
>>> 'b' == ('a' or 'b')
```

```
>>> 'a' == ('a' and 'b')
>>> 'b' == ('a' and 'b')
```

And this will be the unintuitive result:

```
>>> 'a' == ('a' or 'b')
True
>>> 'b' == ('a' or 'b')
False
>>> 'a' == ('a' and 'b')
False
>>> 'b' == ('a' and 'b')
True
```

At this point, the `and` and `or` operators seem to be broken. It doesn't make sense that, for the first two expressions, `'a'` is equivalent to `'a' or 'b'` while `'b'` is not. Furthermore, it doesn't make any sense that `'b'` is equivalent to `'a' and 'b'`. After examining what the interpreter does with boolean operators, these results do in fact exactly what you are asking of them, it's just not the same as what you think you are asking.

When the Python interpreter looks at an `or` expression, it takes the first statement and checks to see if it is true. If the first statement is true, then Python returns that object's value without checking the second statement. This is because for an `or` expression, the whole thing is true if one of the values is true; the program does not need to bother with the second statement. On the other hand, if the first value is evaluated as false Python checks the second half and returns that value. That second half determines the truth value of the whole expression since the first half was false. This "laziness" on the part of the interpreter is called "short circuiting" and is a common way of evaluating boolean expressions in many programming languages.

Similarly, for an `and` expression, Python uses a short circuit technique to speed truth value evaluation. If the first statement is false then the whole thing must be false, so it returns that value. Otherwise if the first value is true it checks the second and returns that value.

One thing to note at this point is that the boolean expression returns a value indicating `True` or `False`, but that Python considers a number of different things to have a truth value assigned to them. To check the truth value of any given object `x`, you can use the function `bool(x)` to see its truth value. Below is a table with examples of the truth values of various objects:

True	False
True	False
1	0
Numbers other than zero	The string 'None'
Nonempty strings	Empty strings
Nonempty lists	Empty lists
Nonempty dictionaries	Empty dictionaries

Now it is possible to understand the perplexing results we were getting when we tested those boolean expressions before. Let's take a look at what the interpreter "sees" as it goes through that code:

First case:

```

»> 'a' == ('a' or 'b') # Look at parentheses first, so evaluate expression "('a' or 'b')"
```

'a' is a nonempty string, so the first value is

True

Return that first value: 'a'

```

»> 'a' == 'a' # the string 'a' is equivalent to the string 'a', so expression
is True
True
```

Second case:

```

»> 'b' == ('a' or 'b') # Look at parentheses first, so evaluate expression "('a' or 'b')"
```

'a' is a nonempty string, so the first value is

True

Return that first value: 'a'

```

»> 'b' == 'a' # the string 'b' is not equivalent to the string 'a', so
expression is False
False
```

Third case:

```

»> 'a' == ('a' and 'b') # Look at parentheses first, so evaluate expression "('a' and 'b')"
```

'a' is a nonempty string, so the first value is

True, examine second value

'b' is a nonempty string, so second value is True

Return that second value as result of whole

expression: 'b'

```

»> 'a' == 'b' # the string 'a' is not equivalent to the string 'b', so
expression is False
False
```

Fourth case:

```

»> 'b' == ('a' and 'b') # Look at parentheses first, so evaluate expression "('a' and 'b')"
```

'a' is a nonempty string, so the first value is

True, examine second value

'b' is a nonempty string, so second value is True

Return that second value as result of whole

expression: 'b'

```

»> 'b' == 'b' # the string 'b' is equivalent to the string 'b', so expression
is True
True
```

So Python was really doing its job when it gave those apparently bogus results. As mentioned previously, the important thing is to recognize what value your boolean expression will return when it is evaluated, because it isn't always obvious.

Going back to those initial expressions, this is how you would write them out so they behaved in a way that you want:

```
>>> 'a' == 'a' or 'a' == 'b'
True
>>> 'b' == 'a' or 'b' == 'b'
True
>>> 'a' == 'a' and 'a' == 'b'
False
>>> 'b' == 'a' and 'b' == 'b'
False
```

When these comparisons are evaluated they return truth values in terms of True or False, not strings, so we get the proper results.

12.0.2 Examples

password1.py

```
## This program asks a user for a name and a password.
# It then checks them to make sure that the user is allowed in.

name = input("What is your name? ")
password = input("What is the password? ")
if name == "Josh" and password == "Friday":
    print("Welcome Josh")
elif name == "Fred" and password == "Rock":
    print("Welcome Fred")
else:
    print("I don't know you.")
```

Sample runs

```
What is your name? Josh
What is the password? Friday
Welcome Josh
```

```
What is your name? Bill
What is the password? Money
I don't know you.
```

12.0.3 Exercises

Write a program that has a user guess your name, but they only get 3 chances to do so until the program quits.

Solution

```
print("Try to guess my name!")
count = 1
name = "guilherme"
guess = input("What is my name? ")
```